HttpListenerStepByStep

Подготовка frontend и ES-модули JavaScript

Пошаговый учебный туториал

Модель · редактор · drag-and-drop · resize · export/import

По текущим исходникам проекта и плану развития с PostgreSQL.

Первое занятие: подготовка frontend
Второй проход: разделение на модули
Полные варианты HTML, CSS и JS приложены отдельно.

Обновлено 3 октября 2026 года

Как провести занятие

Этот туториал разворачивает подготовительные исправления со страницы 3 исходного плана и шаг 1 «Зафиксировать модель и HTTP-контракт». Он опирается на текущие файлы HttpListenerStepByStep, просмотренные 3 октября 2026 года.

Результат: стикер создаётся, редактируется, перемещается и меняет размер. Данные живут в JavaScript-модели. После завершения действия получаем объект, который позднее можно отправить на сервер. После перезагрузки страницы стикеры пока исчезают: сохранение в PostgreSQL будет отдельным этапом.

Продолжительность: 90 минут при разборе примеров; при самостоятельном написании кода удобнее разделить материал на два занятия.

- 0-10 минут: воспроизвести проблемы исходника и обсудить модель.
- 10-25 минут: поправить HTML, размеры стикера и размещение доски.
- 25-45 минут: отделить DOM от данных и завершать редактор текста.
- 45-70 минут: исправить drag-and-drop и resize.
- 70-90 минут: проверить ограничения, разобрать HTTP-контракт.

Что понадобится: текущий проект, браузер, инструменты разработчика; JavaScript-функции, DOM-события, объекты и async/await. На этом занятии не нужны PostgreSQL, Npgsql и новые зависимости.

Читаем материал по порядку. Разделы 2-13 собирают первый вариант script.js, раздел 16 переносит его в ES-модули через export/import. Папка examples содержит первый вариант, examples-modules - итоговый вариант с public/js. На второй проход выделите ещё 45-60 минут.

В первом проходе Program.cs и .csproj менять не требуется. В разделе 16 поправим шаблон копирования public в .csproj для вложенных JS-файлов. Используем существующий GET /quote/random. Материал раздела 16 соответствует шагу 6 исходного плана.

1. Что исправляем в исходнике

В public/script.js createSticker одновременно запрашивает цитату, создаёт HTML и подключает взаимодействия. Состояние хранится в DOM. Перед сохранением в БД надо явно выделить данные и исправить поведение интерфейса.

- addDragAndDrop: clientX/clientY относятся к окну, а style.left/top - к доске. При смещении доски стикер прыгает.
- addStickerText: textarea остаётся после редактирования; завершение редактора не возвращает обычный текст.
- addResizeFeature вызывается повторно из редактора. Это создаёт новые ручки и обработчики.
- mousedown ручки всплывает до стикера: resize может одновременно запускать drag.
- innerHTML интерпретирует цитату и пользовательский ввод как HTML.
- .sticker имеет padding: 10px без border-box. CSS-ширина 200px даёт внешнюю ширину 220px.
- Размер шрифта умножается на коэффициент на каждом mousemove и постепенно искажается.

Вместо чтения данных из innerHTML вводим цепочку:

```
модель -> renderSticker -> DOM
действие пользователя -> обновление модели
завершение действия -> onChange -> будущий PUT
```

До правок добавьте стикер, дважды отредактируйте его и потяните за угол. Затем в инструментах разработчика посмотрите число .resize-handle и внешние размеры стикера. Повторите те же проверки после занятия.

Все примеры ниже заменяют старую реализацию, а не добавляются рядом с прежними функциями. Особенно важно убрать старые addStickerText, adjustFontSize, addDragAndDrop и addResizeFeature, чтобы не смешать обработчики.

2. Вводим модель и единые ограничения

Файл: public/script.js. Начните новый вариант файла с этого блока. В модели доски есть id, title, createdAt и коллекция stickers. Стикер содержит id, boardId, text, x, y, width, height, createdAt.

Здесь boardModel.id равен null: настоящую доску ещё не создаёт сервер. Локальные UUID стикеров нужны только для учебного DOM. При подключении POST сервер будет возвращать окончательные id и boardId. createdAt тоже придёт с сервера.

```
// 1. Модель и границы значений
const LIMITS = { minSize: 80, maxValue: 10000, maxText: 2000 };
const board = document.getElementById('board');
const boardModel = {
    id: null, title: 'Учебная доска', createdAt: null, stickers: []
};

function clampPixel(value, min, max = LIMITS.maxValue) {
    return Math.min(max, Math.max(min, Math.round(value)));
}

function toStickerPayload(model) {
    return {
        text: model.text, x: model.x, y: model.y,
        width: model.width, height: model.height
    };
}

function handleStickerChange(model) {
    // Позже здесь будет PUT. Сейчас проверяем контракт в консоли.
    console.log('Изменение завершено:', toStickerPayload(model));
}
```

x и y - целые пиксели от внутреннего начала доски, в диапазоне 0..10000. width и height - внешние размеры 80..10000. Текст ограничен 2000 символами по принятому контракту; для title позже проверяем trim(), непустое значение и длину до 120.

toStickerPayload явно выбирает все пять изменяемых полей. Шрифт, CSS-классы и DOM-узлы не попадают в JSON. Это пригодится для PUT, который заменяет все эти поля сразу.

Проверка в консоли: clampPixel(-10, 0) даёт 0; clampPixel(79, 80) даёт 80; clampPixel(10001, 0) даёт 10000; clampPixel(10.7, 0) даёт 11.

3. Уточняем HTML и размещение панели

Файл: public/index.html. Ниже полный вариант. Сохраняем id board, addSticker и addText, а также исходные SVG. Выносим панель из доски: это самостоятельный фиксированный элемент интерфейса.

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Учебная доска</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header><h2>Учебная доска</h2></header>
    <main>
        <div class="board" id="board" aria-label="Доска"></div>
    </main>
    <div class="toolbar">
        <button id="addSticker" class="toolbar-button"
                type="button" aria-label="Добавить стикер">
            <img src="sticker.svg" alt="">
        </button>
        <button id="addText" class="toolbar-button" type="button"
                disabled title="Отдельный тип текста добавим позже">
            <img src="text.svg" alt="Текст">
        </button>
    </div>
    <footer><h2>HttpListenerStepByStep</h2></footer>
    <script src="script.js"></script>
</body>
</html>
```

Кнопку addText оставляем отключённой с объяснением. В исходном плане отдельный тип «текст» отложен на следующий учебный этап; сейчас работаем только со стикерами.

Обычный script размещён после HTML, поэтому элементы уже существуют при запуске. type="module" пока не добавляем. Проверьте, что script.js подключён ровно один раз.

4. Делаем доску прокручиваемой

Файл: public/styles.css. Замените старые стили приведённым вариантом: первая половина ниже, продолжение в следующем разделе. Доска занимает пространство между шапкой и подвалом; её начало теперь не совпадает с началом окна. Это сразу позволяет проверить правильность координат.

```
* { box-sizing: border-box; }
html, body { margin: 0; height: 100%; }
body {
    font-family: Arial, sans-serif;
    display: grid;
    grid-template-rows: 48px minmax(0, 1fr) 40px;
}
header, footer {
    display: flex;
    align-items: center;
    justify-content: center;
}
header { background: #d8a668; }
footer { background: #69969f; }
h2 { margin: 0; font-size: clamp(16px, 2.5vw, 28px); }
main { min-height: 0; min-width: 0; }
.board {
    position: relative;
    width: 100%;
    height: 100%;
    padding: 0;
    overflow: auto;
    background-color: white;
    background-image: linear-gradient(#eee 1px, transparent 1px);
    background-size: 100% 24px;
}
.toolbar {
    position: fixed;
    bottom: 56px;
    left: 50%;
    transform: translateX(-50%);
    display: flex;
    gap: 12px;
    z-index: 1000;
    padding: 8px;
    border: 2px solid black;
    border-radius: 15px;
    background: #f9f9f9;
}
.toolbar-button {
    border: none;
    background: none;
    cursor: pointer;
}
.toolbar-button:disabled { opacity: .5; cursor: default; }
.toolbar-button img { width: 36px; height: 36px; }
```

minmax(0, 1fr) и min-height: 0 позволяют средней области уменьшаться вместе с окном. overflow: auto сохраняет доступ к стикерам за пределами видимой области. Координаты не пересчитываем при resize окна.

В CSS нет padding у доски: начало абсолютного позиционирования не получает дополнительного смещения. Толщину границы, если она появится, учитываем в getBoardPoint.

5. Размер, текст и одна ручка

Продолжение public/styles.css. box-sizing: border-box делает width/height внешними размерами с учётом padding и границы. Модель width: 200 должна соответствовать offsetWidth: 200.

```
.sticker {
    position: absolute;
    box-sizing: border-box;
    min-width: 80px;
    min-height: 80px;
    padding: 12px;
    border-radius: 10px;
    background: #ffe58a;
    box-shadow: 3px 3px 5px #0006;
    overflow: hidden;
    cursor: grab;
}
.sticker-text, .input {
    width: 100%;
    height: 100%;
    margin: 0;
    padding: 0;
    border: none;
    font-family: inherit;
    font-size: 20px;
    line-height: 1.2;
    color: inherit;
    text-align: center;
    white-space: pre-wrap;
    overflow-wrap: anywhere;
    overflow: auto;
}
.input {
    display: block;
    resize: none;
    background: inherit;
    cursor: text;
}
.resize-handle {
    position: absolute;
    bottom: 0;
    right: 0;
    width: 12px;
    height: 12px;
    background: gray;
    cursor: nwse-resize;
}
```

white-space: pre-wrap сохраняет переносы строк. Если 2000 символов не помещаются даже при минимальном шрифте, содержимое можно прокрутить. Мы не обещаем вместить любой текст в квадрат 80 × 80.

Проверка: стикер 200 × 200 имеет именно такие внешние размеры. Текст с длинным словом и несколькими строками остаётся доступным. Ручка находится в правом нижнем углу.

6. Строим DOM из модели

Файл: public/script.js, после блока модели. renderSticker создаёт три элемента: контейнер, текст и единственную ручку. Ручку создаём один раз за жизнь DOM-стикера. Редактор потом заменяет только текстовый узел.

```
// 2. Текст и представление
function fitText(element, min = 12, max = 36) {
    for (let size = max; size >= min; size--) {
        element.style.fontSize = `${size}px`;
        if (element.scrollHeight <= element.clientHeight &&
            element.scrollWidth <= element.clientWidth) return;
    }
}

function applyGeometry(sticker, model) {
    sticker.style.left = `${model.x}px`;
    sticker.style.top = `${model.y}px`;
    sticker.style.width = `${model.width}px`;
    sticker.style.height = `${model.height}px`;
}

function renderSticker(model, onChange) {
    const sticker = document.createElement('div');
    sticker.className = 'sticker';
    sticker.dataset.id = model.id;

    const text = document.createElement('div');
    text.className = 'sticker-text';
    text.textContent = model.text;

    const handle = document.createElement('div');
    handle.className = 'resize-handle';
    sticker.append(text, handle);
    applyGeometry(sticker, model);
    board.appendChild(sticker); // Нужен layout перед fitText.
    fitText(text);

    sticker.addEventListener('dblclick', event => {
        if (event.target.closest('.resize-handle')) return;
        editStickerText(sticker, text, model, onChange);
    });
    addDragAndDrop(sticker, model, onChange);
    addResizeFeature(sticker, handle, text, model, onChange);
    return sticker;
}
```

fitText каждый раз начинает с max и уменьшает размер до первого подходящего. Поэтому шрифт не накапливает ошибку предыдущих mousemove. Этот размер вычисляется из текста и геометрии, а не хранится в модели.

fitText вызываем после добавления в board: у отключённого от страницы элемента нельзя рассчитывать на реальные размеры. Для строки пример textContent показывает угловые скобки и буквы буквально, что соответствует будущему хранению обычного текста.

Проверка: model.text и .sticker-text.textContent совпадают; у стикера одна .resize-handle. При повторной отрисовке одного набора данных сначала следует удалять предыдущие DOM-стикеры, иначе их будет два.

7. Завершаем редактирование текста

Файл: public/script.js. Удаляем прежнюю addStickerText и добавляем этот редактор. Двойной клик заменяет только текст на textarea; blur возвращает обычный текстовый элемент.

```
// 3. Редактор с завершением по blur
function editStickerText(sticker, text, model, onChange) {
    if (sticker.querySelector('textarea')) return;
    const originalText = model.text;
    const input = document.createElement('textarea');
    input.className = 'input';
    input.maxLength = LIMITS.maxText;
    input.value = originalText;
    input.setAttribute('aria-label', 'Текст стикера');
    text.replaceWith(input); // Ручка остается в DOM.
    input.focus();
    input.setSelectionRange(input.value.length, input.value.length);
    fitText(input);
    input.addEventListener('input', () => fitText(input));

    let cancelled = false;
    input.addEventListener('keydown', event => {
        if (event.key === 'Escape') {
            event.preventDefault();
            cancelled = true;
            input.blur();
        } else if (event.key === 'Enter' && event.ctrlKey) {
            event.preventDefault();
            input.blur();
        }
    });
    input.addEventListener('blur', () => {
        model.text = cancelled ? originalText : input.value;
        text.textContent = model.text;
        input.replaceWith(text);
        fitText(text);
        if (model.text !== originalText) onChange({ ...model });
    }, { once: true });
}
```

Enter создаёт новую строку, Ctrl+Enter завершает ввод, Escape отменяет изменения. Пустую строку сохраняем как пустую: автоматическое «Стикер пуст» изменяло бы данные за пользователя.

onChange вызывается один раз при завершении, только если текст изменился. На каждый символ запросов не будет. Обработчик blur одноразовый; глобальный document.click для каждого редактора больше не нужен.

Проверка: введите две строки и нажмите вне стикера. textarea исчезла, текст в модели обновился, ручка осталась одна. Повторите трижды; число ручек не растёт. Escape возвращает прежний текст.

8. Согласуем две системы координат

Файл: public/script.js. clientX/clientY и getBoundingClientRect относятся к видимой области окна. left/top стикера относятся к доске. Поэтому вычитаем начало доски и границу, затем добавляем её прокрутку.

```
// 4. Перевод координат окна в координаты доски
function getBoardPoint(event) {
    const rect = board.getBoundingClientRect();
    return {
        x: event.clientX - rect.left - board.clientLeft
            + board.scrollLeft,
        y: event.clientY - rect.top - board.clientTop
            + board.scrollTop
    };
}

// 5. Общая подписка на движение мыши до окончания действия
function trackMouse(onMove, onEnd) {
    function finish() {
        document.removeEventListener('mousemove', onMove);
        document.removeEventListener('mouseup', finish);
        window.removeEventListener('blur', finish);
        onEnd();
    }
    document.addEventListener('mousemove', onMove);
    document.addEventListener('mouseup', finish);
    window.addEventListener('blur', finish);
}
```

Числовой пример: rect.left = 100, clientLeft = 2, scrollLeft = 300, clientX = 150. Координата точки внутри содержимого доски: 150 - 100 - 2 + 300 = 348. Если точка захвата отстоит от левого края стикера на 20px, новая x будет 328.

trackMouse подключает движение к document: курсор может уйти за пределы стикера. mouseup снимает подписки; уход фокуса окна также завершает действие. Координаты доски пересчитываются при каждом движении, включая изменение scrollLeft/scrollTop.

Пример рассчитан на мышь и обычную доску без CSS transform/масштабирования. Touch, Pointer Events и автопрокрутка при приближении к краю - отдельные упражнения.

9. Исправляем drag-and-drop

Файл: public/script.js. Меняем старую addDragAndDrop целиком. Начальное смещение курсора внутри стикера вычисляем в координатах доски и сохраняем до mouseup.

```
// 6. Drag-and-drop
function addDragAndDrop(sticker, model, onChange) {
    sticker.addEventListener('mousedown', event => {
        if (event.button !== 0 ||
            event.target.closest('textarea, .resize-handle') ||
            sticker.querySelector('textarea')) return;
        event.preventDefault();
        const start = { x: model.x, y: model.y };
        const point = getBoardPoint(event);
        const offsetX = point.x - model.x;
        const offsetY = point.y - model.y;
        sticker.style.cursor = 'grabbing';

        trackMouse(moveEvent => {
            const next = getBoardPoint(moveEvent);
            model.x = clampPixel(next.x - offsetX, 0);
            model.y = clampPixel(next.y - offsetY, 0);
            applyGeometry(sticker, model);
        }, () => {
            sticker.style.cursor = 'grab';
            if (model.x !== start.x || model.y !== start.y) {
                onChange({ ...model });
            }
        });
    });
}
```

Во время движения обновляем модель и DOM. После отпускания мыши передаём снимок модели через onChange. Если координаты не изменились, callback не нужен.

Правая кнопка, textarea и ручка resize не запускают drag. Во время открытого редактора стикер целиком не перемещаем. preventDefault предотвращает выделение текста при обычном перетаскивании.

Проверка: захватите стикер за середину. Он не должен прыгнуть под курсор. Перетащите влево за начало доски - x станет 0. Переместите далеко вправо, прокрутите доску и повторите действие: смещение курсора сохраняется. Один законченный drag даёт одну запись в консоли.

10. Исправляем resize

Файл: public/script.js. Функция получает уже созданную ручку. Она не создаёт новые элементы, поэтому редактору не нужно вызывать её повторно.

```
// 7. Resize с одной ручкой и независимыми событиями
function addResizeFeature(sticker, handle, text, model, onChange) {
    handle.addEventListener('mousedown', event => {
        event.stopPropagation();
        if (event.button !== 0 || sticker.querySelector('textarea')) {
            return;
        }
        event.preventDefault();
        const point = getBoardPoint(event);
        const start = {
            width: model.width, height: model.height,
            x: point.x, y: point.y
        };

        trackMouse(moveEvent => {
            const next = getBoardPoint(moveEvent);
            model.width = clampPixel(
                start.width + next.x - start.x, LIMITS.minSize);
            model.height = clampPixel(
                start.height + next.y - start.y, LIMITS.minSize);
            applyGeometry(sticker, model);
            fitText(text);
        }, () => {
            if (model.width !== start.width ||
                model.height !== start.height) onChange({ ...model });
        });
    });
}
```

stopPropagation выполняется до остальных проверок и не позволяет mousedown ручки попасть в drag стикера. Размеры вычисляем от начальной геометрии; текст подбираем заново по текущим габаритам.

Ширина и высота независимы. Каждую округляем и ограничиваем от 80 до 10000. В модели не должно быть отрицательного, дробного или бесконечного размера. Эта функция получает только числа из событий браузера; будущий JSON сервер обязан валидировать отдельно.

Проверка: потяните ручку внутрь - размеры остановятся на 80 × 80. При resize x/y остаются прежними. Увеличьте стикер и уменьшите обратно: шрифт не разрастается с каждым движением. В консоли одна запись после mouseup.

11. Получаем начальный текст

Файл: public/script.js. Существующий маршрут /quote/random сохраняем. В исходнике обрабатывался неуспешный HTTP-статус, но разрыв сети мог прервать создание стикера исключением.

```
// 8. Создание локального стикера с необязательной цитатой
async function getInitialText() {
    try {
        const response = await fetch('/quote/random', {
            signal: AbortSignal.timeout(5000)
        });
        if (!response.ok) throw new Error('Цитата недоступна');
        const data = await response.json();
        if (typeof data.quote !== 'string' ||
            typeof data.author !== 'string') {
            throw new Error('Неверный формат цитаты');
        }
        return `${data.quote} (c) ${data.author}`
            .slice(0, LIMITS.maxText);
    } catch {
        return 'Новый стикер';
    }
}
```

Проверяем HTTP-статус и форму JSON. Ошибка, не-JSON ответ или ожидание дольше пяти секунд дают текст «Новый стикер». Длину цитаты ограничиваем перед созданием модели.

AbortSignal.timeout используется в современном браузере; при отсутствии этого API ошибка также попадёт в catch и стикер будет создан с запасным текстом. Для отдельного урока по совместимости можно добавить AbortController и таймер.

Это единственный HTTP-запрос текущего frontend. GET /api/boards и POST/PUT стикеров здесь ещё не вызываем. Когда будут загружены стикеры из БД, для них не нужно повторно запрашивать цитаты.

Проверка: при доступном /quote/random создаётся цитата. При 500, неверном JSON или сетевой ошибке создаётся «Новый стикер», а кнопка становится доступна снова.

12. Создаём модель перед отображением

Файл: public/script.js. Продолжение предыдущего блока. Случайное размещение используем в видимой части доски; на маленьком экране доступное пространство ограничиваем снизу нулём.

```
async function createSticker() {
    const button = document.getElementById('addSticker');
    button.disabled = true;
    try {
        const text = await getInitialText();
        const width = 200;
        const height = 200;
        const model = {
            // Это временный локальный id; позже id придет из POST.
            id: crypto.randomUUID(), boardId: boardModel.id,
            text,
            x: clampPixel(board.scrollLeft + Math.random() *
                Math.max(0, board.clientWidth - width), 0),
            y: clampPixel(board.scrollTop + Math.random() *
                Math.max(0, board.clientHeight - height), 0),
            width, height, createdAt: null
        };
        boardModel.stickers.push(model);
        renderSticker(model, handleStickerChange);
    } finally {
        button.disabled = false;
    }
}
```

Math.max(0, clientWidth - width) предотвращает отрицательное случайное смещение, если доска уже 200px. Так как размер стикера больше экрана, его часть будет доступна прокруткой. При прокрученной доске добавляем текущий scrollLeft/scrollTop.

Модель добавляем в boardModel.stickers, затем вызываем renderSticker. DOM и обработчики используют тот же объект; callback получает копию на момент завершения действия. Блок finally возвращает кнопку в рабочее состояние.

Локальный id не означает, что объект сохранён. На будущем этапе алгоритм поменяется: POST -> ответ с серверным id -> добавление в модель -> отображение.

13. Привязываем события и запускаем

Файл: public/script.js, завершение файла. Старые window.onload/window.onresize убираем. Подбор шрифта шапки и подвала теперь делает CSS clamp; прежние adjustHeadersFontSize и adjustFontSizeHeaders не нужны.

```
// 9. Запуск без перезаписи window.onload/window.onresize
document.getElementById('addSticker')
    .addEventListener('click', createSticker);
window.addEventListener('resize', () => {
    board.querySelectorAll('.sticker-text, .input').forEach(element => {
        fitText(element);
    });
});
```

Начальный fitText стикера вызывается из renderSticker. Resize окна меняет только вычисляемый шрифт. x/y/width/height моделей остаются прежними, что соответствует исходному плану.

Чтобы проверить итоговые файлы в текущем проекте:

1. Сохраните свою рабочую копию трёх public-файлов.
2. Перенесите index.html, styles.css и script.js из examples в HttpListenerStepByStep/public, заменив прежние версии. SVG уже есть в public.
3. Запустите сервер из каталога HttpListenerStepByStep. Текущий Program.cs ищет public относительно рабочей папки.

```
Set-Location HttpListenerStepByStep
dotnet run
```

Откройте http://localhost:5000/, затем Console и Network. При повторном запуске страницы выполните обновление без кэша, если браузер показывает старый script.js.

Не запускайте для этого урока сервер из произвольной папки: перенос поиска статики на AppContext.BaseDirectory относится к следующему серверному этапу плана. Открытие через file:// тоже не воспроизводит серверный маршрут цитат.

14. Фиксируем будущий HTTP-контракт

На этом занятии записываем контракт, а реализуем маршруты в шаге 5 исходного плана. Все пути с id используют UUID. Для создания доски отправляем {"title":"Моя доска"}; для стикера - результат toStickerPayload.

```
POST   /api/boards
       -> 201, { id, title }, Location
GET    /api/boards/{boardId}
       -> 200, { id, title, stickers: [] }
POST   /api/boards/{boardId}/stickers
       -> 201, стикер с id и boardId, Location
PUT    /api/boards/{boardId}/stickers/{stickerId}
       -> 204, без тела
DELETE /api/boards/{boardId}/stickers/{stickerId}
       -> 204, без тела
DELETE /api/boards/{boardId}
       -> 204, без тела
```

PUT получает все пять изменяемых полей, включая поля без изменений. Пример будущего тела:

```
{
  "text": "План занятия\nИсправить resize",
  "x": 120,
  "y": 60,
  "width": 240,
  "height": 180
}
```

Пустая существующая доска возвращает 200 и пустой массив, отсутствующая - 404. Неизвестный стикер при PUT/DELETE и повторное удаление также дают 404. При 204 браузер не должен вызывать response.json().

Ошибки: 400 - неверный JSON, UUID или поля; 404 - нет объекта; 405 - метод не разрешён (с Allow); 415 - неподдерживаемый Content-Type; 500 - непредвиденный сбой. Общая форма:

```
{"error":{"code":"validation_error",
          "message":"Некорректные размеры стикера"}}
```

Клиентские ограничения удобны для интерфейса, но сервер и PostgreSQL позже проверят их независимо. UUID гостевой доски даёт доступ знающему его человеку; localStorage будет хранить список известных UUID и активный id, а не владение и не тексты стикеров.

15. Проверяем результат и закрепляем

Приёмка проводится после сборки всех фрагментов, через существующий сервер.

1. Создайте два стикера: у каждого отдельный локальный id и одна ручка. На узком экране x/y не становятся отрицательными.
2. Введите кириллицу, перенос строки и пример. Текст отображается буквально. Пустая строка тоже сохраняется в модели.
3. Завершите ввод кликом снаружи и Ctrl+Enter. У textarea происходит обратная замена. Escape отменяет текст и не вызывает изменение.
4. Трижды откройте редактор. Ручек по-прежнему по одной, событие завершения не дублируется.
5. Перетаскивайте доску со смещением и прокруткой. x/y совпадают с положением стикера; при движении ручки x/y не изменяются.
6. Уменьшите стикер ниже минимума. Модель и DOM останавливаются на 80 × 80. Размер 200 × 200 совпадает с offsetWidth/offsetHeight.
7. Проверьте консоль: одна запись после изменённого drag/resize/текста, ни одной после действия без изменения. Во время движения записей нет.
8. Измените размер окна. Сохранённые в модели координаты и размеры не меняются, доступ за пределами окна обеспечивается прокруткой.
9. Смоделируйте неуспешный запрос цитаты. Создание использует запасной текст; кнопка снова доступна.
10. Перезагрузите страницу: стикеры исчезают. Это ожидаемое состояние до этапа PostgreSQL, а не результат сохранения.

Контрольные вопросы: почему x/y не равны clientX/clientY? Почему width - внешний размер? Почему ручку не создаём в редакторе? Почему fontSize не поле модели? Какие пять полей нужны для PUT?

Практика: добавьте в интерфейс небольшой вывод JSON выбранного стикера через toStickerPayload; придумайте проверку, доказывающую независимость данных от HTML. Не используйте innerHTML для восстановления текста.

Учебный коммит после ручной проверки: «Подготовить frontend: модель стикера, редактор, drag и resize». Коммит выполняет преподаватель или студент в своей рабочей копии.

16. Разделяем JavaScript через export/import

Это второй проход по уже работающему приложению. Сначала закончите разделы 1-15 и проверьте script.js; затем переносите функции в public/js. Выделите ещё 45-60 минут на перенос и проверку. Поведение доски, текст и ограничения должны остаться прежними.

Итоговая структура:

```
public/
  index.html
  styles.css
  sticker.svg
  text.svg
  js/
    main.js
    model.js
    typography.js
    interactions.js
    sticker-view.js
    api.js
    board-controller.js
```

export делает объявление доступным для других модулей. import получает именованный экспорт. Имя внутри фигурных скобок должно совпадать с именем экспорта:

```
// model.js
export function clampPixel(value, min, max = 10000) {
    return Math.min(max, Math.max(min, Math.round(value)));
}

// другой модуль
import { clampPixel } from './model.js';
```

В реальном model.js используем LIMITS из следующего раздела, а не создаём второй clampPixel. Это только пример синтаксиса. Именованные экспорты позволяют одному файлу предоставлять несколько функций. default export для этого урока не нужен.

Пути считаются относительно импортирующего JS-файла. Указываем ./ и расширение .js: './model.js'. Импорт './Model.js' может сломаться на системе с чувствительностью к регистру. Функции модуля не становятся свойствами window; вызываем их через import.

Зависимости направлены от main к контроллеру и представлению. Ни один модуль не импортирует main. Представление и взаимодействия получают board, модель и callback параметрами; сетевые запросы остаются в api.js.

```
main -> board-controller -> api -> model
main -> typography
board-controller -> sticker-view -> interactions
sticker-view -> typography, model
interactions -> typography, model
board-controller -> model
```

Папка examples сохраняет первый вариант с одним script.js. Папка examples-modules содержит окончательный вариант этого занятия. Не подключайте оба варианта одновременно.

16.1. model.js и typography.js: простые экспорты

Сначала переносим независимые функции. model.js не обращается к DOM и не хранит активную доску. Он задаёт общие ограничения и форму будущего HTTP-запроса. Полный файл:

```
// 1. Модель и границы значений
export const LIMITS = { minSize: 80, maxValue: 10000, maxText: 2000 };

export function clampPixel(value, min, max = LIMITS.maxValue) {
    return Math.min(max, Math.max(min, Math.round(value)));
}

export function toStickerPayload(model) {
    return {
        text: model.text, x: model.x, y: model.y,
        width: model.width, height: model.height
    };
}
```

typography.js получает элемент параметром и меняет только вычисляемый размер шрифта. Полный файл:

```
// 2. Текст и представление
export function fitText(element, min = 12, max = 36) {
    for (let size = max; size >= min; size--) {
        element.style.fontSize = `${size}px`;
        if (element.scrollHeight <= element.clientHeight &&
            element.scrollWidth <= element.clientWidth) return;
    }
}
```

export добавляем к объявлению, тело функции сохраняем. Импорт не копирует код функции в другой файл: модуль предоставляет объявленный экспорт.

Проверка в Console браузера после запуска модульного варианта:

```
const helpers = await import('/js/model.js');
helpers.clampPixel(79, 80); // 80
```

Обычный вызов clampPixel(...) в консоли уже не обращается к модульной функции: она не глобальная. Динамический import здесь используется только для учебной проверки; в исходниках оставляем статические импорты.

16.2. interactions.js: явно передаём доску

Перенесите applyGeometry, getBoardPoint, trackMouse, addDragAndDrop и addResizeFeature из первого варианта в public/js/interactions.js. Уберите зависимость от глобальной переменной board.

Главная правка: добавляем board первым аргументом в getBoardPoint, addDragAndDrop и addResizeFeature. Все вызовы getBoardPoint внутри обработчиков тоже получают board. applyGeometry разместили здесь, чтобы sticker-view и interactions не импортировали друг друга.

Ниже полный файл. Внутренние getBoardPoint и trackMouse не экспортируем: их использует только этот модуль.

```
import { LIMITS, clampPixel } from './model.js';
import { fitText } from './typography.js';

export function applyGeometry(sticker, model) {
    sticker.style.left = `${model.x}px`;
    sticker.style.top = `${model.y}px`;
    sticker.style.width = `${model.width}px`;
    sticker.style.height = `${model.height}px`;
}

// 4. Перевод координат окна в координаты доски
function getBoardPoint(board, event) {
    const rect = board.getBoundingClientRect();
    return {
        x: event.clientX - rect.left - board.clientLeft
            + board.scrollLeft,
        y: event.clientY - rect.top - board.clientTop
            + board.scrollTop
    };
}

// 5. Общая подписка на движение мыши до окончания действия
function trackMouse(onMove, onEnd) {
    function finish() {
        document.removeEventListener('mousemove', onMove);
        document.removeEventListener('mouseup', finish);
        window.removeEventListener('blur', finish);
        onEnd();
    }
    document.addEventListener('mousemove', onMove);
    document.addEventListener('mouseup', finish);
    window.addEventListener('blur', finish);
}
```

16.2. interactions.js: drag и resize

```
// 6. Drag-and-drop
export function addDragAndDrop(board, sticker, model, onChange) {
    sticker.addEventListener('mousedown', event => {
        if (event.button !== 0 ||
            event.target.closest('textarea, .resize-handle') ||
            sticker.querySelector('textarea')) return;
        event.preventDefault();
        const start = { x: model.x, y: model.y };
        const point = getBoardPoint(board, event);
        const offsetX = point.x - model.x;
        const offsetY = point.y - model.y;
        sticker.style.cursor = 'grabbing';

        trackMouse(moveEvent => {
            const next = getBoardPoint(board, moveEvent);
            model.x = clampPixel(next.x - offsetX, 0);
            model.y = clampPixel(next.y - offsetY, 0);
            applyGeometry(sticker, model);
        }, () => {
            sticker.style.cursor = 'grab';
            if (model.x !== start.x || model.y !== start.y) {
                onChange({ ...model });
            }
        });
    });
}

// 7. Resize с одной ручкой и независимыми событиями
export function addResizeFeature(
    board, sticker, handle, text, model, onChange) {
    handle.addEventListener('mousedown', event => {
        event.stopPropagation();
        if (event.button !== 0 || sticker.querySelector('textarea')) {
            return;
        }
        event.preventDefault();
        const point = getBoardPoint(board, event);
        const start = {
            width: model.width, height: model.height,
            x: point.x, y: point.y
        };

        trackMouse(moveEvent => {
            const next = getBoardPoint(board, moveEvent);
            model.width = clampPixel(
                start.width + next.x - start.x, LIMITS.minSize);
            model.height = clampPixel(
                start.height + next.y - start.y, LIMITS.minSize);
            applyGeometry(sticker, model);
            fitText(text);
        }, () => {
            if (model.width !== start.width ||
                model.height !== start.height) onChange({ ...model });
        });
    });
}
```

Проверка: экспортируются только applyGeometry, addDragAndDrop и addResizeFeature. При переносе найдите все вызовы getBoardPoint: пропущенный board приведёт к ошибке или неверным координатам.

16.3. sticker-view.js: импортируем зависимости

Представление импортирует ограничения, подбор шрифта и функции взаимодействия. renderSticker теперь получает board явно; редактор остаётся внутренней функцией. Полный файл:

```
import { LIMITS } from './model.js';
import { fitText } from './typography.js';
import { applyGeometry, addDragAndDrop,
    addResizeFeature } from './interactions.js';

export function renderSticker(board, model, onChange) {
    const sticker = document.createElement('div');
    sticker.className = 'sticker';
    sticker.dataset.id = model.id;

    const text = document.createElement('div');
    text.className = 'sticker-text';
    text.textContent = model.text;

    const handle = document.createElement('div');
    handle.className = 'resize-handle';
    sticker.append(text, handle);
    applyGeometry(sticker, model);
    board.appendChild(sticker); // Нужен layout перед fitText.
    fitText(text);

    sticker.addEventListener('dblclick', event => {
        if (event.target.closest('.resize-handle')) return;
        editStickerText(sticker, text, model, onChange);
    });
    addDragAndDrop(board, sticker, model, onChange);
    addResizeFeature(board, sticker, handle, text, model, onChange);
    return sticker;
}
```

16.3. sticker-view.js: редактор текста

```
// 3. Редактор с завершением по blur
function editStickerText(sticker, text, model, onChange) {
    if (sticker.querySelector('textarea')) return;
    const originalText = model.text;
    const input = document.createElement('textarea');
    input.className = 'input';
    input.maxLength = LIMITS.maxText;
    input.value = originalText;
    input.setAttribute('aria-label', 'Текст стикера');
    text.replaceWith(input); // Ручка остается в DOM.
    input.focus();
    input.setSelectionRange(input.value.length, input.value.length);
    fitText(input);
    input.addEventListener('input', () => fitText(input));

    let cancelled = false;
    input.addEventListener('keydown', event => {
        if (event.key === 'Escape') {
            event.preventDefault();
            cancelled = true;
            input.blur();
        } else if (event.key === 'Enter' && event.ctrlKey) {
            event.preventDefault();
            input.blur();
        }
    });
    input.addEventListener('blur', () => {
        model.text = cancelled ? originalText : input.value;
        text.textContent = model.text;
        input.replaceWith(text);
        fitText(text);
        if (model.text !== originalText) onChange({ ...model });
    }, { once: true });
}
```

Импортированные addDragAndDrop и addResizeFeature вызываем с board первым аргументом. Представление получает onChange от контроллера, но не знает URL API и не делает fetch. Это позволит позже менять сохранение без переписывания редактора и DOM.

Проверка: редактор сохраняет одну ручку, возвращает текст после blur и сообщает о завершении через callback. В файле нет чтения глобального board и нет импорта board-controller.

16.4. api.js: сетевой запрос отдельно

На этом этапе API-модуль предоставляет только уже работающий запрос цитаты. Будущие GET/POST/PUT досок добавим после реализации серверных маршрутов. Полный public/js/api.js:

```
import { LIMITS } from './model.js';

// 8. Создание локального стикера с необязательной цитатой
export async function getInitialText() {
    try {
        const response = await fetch('/quote/random', {
            signal: AbortSignal.timeout(5000)
        });
        if (!response.ok) throw new Error('Цитата недоступна');
        const data = await response.json();
        if (typeof data.quote !== 'string' ||
            typeof data.author !== 'string') {
            throw new Error('Неверный формат цитаты');
        }
        return `${data.quote} (c) ${data.author}`
            .slice(0, LIMITS.maxText);
    } catch {
        return 'Новый стикер';
    }
}
```

getInitialText асинхронная: контроллер должен вызвать await getInitialText(). Модуль не создаёт DOM и не обращается к кнопкам.

Заготовки с фиктивным сохранением не добавляем. Если HTTP-сохранения ещё нет, callback продолжает писать модель в консоль. board-storage.js для localStorage появится при восстановлении гостевых досок в следующем этапе.

Проверка: при успешном ответе получаем текст цитаты; при 500 и неверном JSON - «Новый стикер». Перенос fetch не должен менять этот сценарий.

16.5. board-controller.js: состояние и команды

Контроллер хранит boardModel в замыкании. main получает только команду createSticker, а представление - callback handleStickerChange. Отдельные переменные boardModel и board больше не нужны на уровне страницы.

Полный public/js/board-controller.js:

```
import { clampPixel, toStickerPayload } from './model.js';
import { getInitialText } from './api.js';
import { renderSticker } from './sticker-view.js';

export function createBoardController(board, button) {
    const boardModel = {
        id: null, title: 'Учебная доска',
        createdAt: null, stickers: []
    };

    function handleStickerChange(model) {
        // Здесь позднее появится PUT с полным телом запроса.
        console.log('Изменение завершено:', toStickerPayload(model));
    }

    async function createSticker() {
        button.disabled = true;
        try {
            const text = await getInitialText();
            const width = 200;
            const height = 200;
            const model = {
                // Это временный локальный id; позже id придет из POST.
                id: crypto.randomUUID(), boardId: boardModel.id,
                text,
                x: clampPixel(board.scrollLeft + Math.random() *
                    Math.max(0, board.clientWidth - width), 0),
                y: clampPixel(board.scrollTop + Math.random() *
                    Math.max(0, board.clientHeight - height), 0),
                width, height, createdAt: null
            };
            boardModel.stickers.push(model);
            renderSticker(board, model, handleStickerChange);
        } finally {
            button.disabled = false;
        }
    }

    return { createSticker };
}
```

createBoardController вызываем один раз для текущей доски. Возвращаемый объект содержит ссылку на функцию; return { createSticker } эквивалентен return { createSticker: createSticker }.

Не создавайте контроллер при каждом клике: иначе коллекция stickers каждый раз начиналась бы заново. В будущем контроллер заменит локальную модель результатом GET и отправит PUT в handleStickerChange.

Проверка: кнопка блокируется на время запроса цитаты и разблокируется через finally. Создание и последующие изменения стикера используют одну и ту же модель.

16.6. main.js, HTML и копирование public/js

main.js является точкой входа: находит элементы, создаёт контроллер и привязывает события. Полный файл:

```
import { createBoardController } from './board-controller.js';
import { fitText } from './typography.js';

const board = document.getElementById('board');
const button = document.getElementById('addSticker');
const controller = createBoardController(board, button);

button.addEventListener('click', controller.createSticker);
window.addEventListener('resize', () => {
    board.querySelectorAll('.sticker-text, .input').forEach(element => {
        fitText(element);
    });
});
```

В public/index.html замените прежнее подключение script.js одной строкой:

```
<script type="module" src="/js/main.js"></script>
```

Старый script.js больше не подключайте. Его можно убрать из public после завершения переноса; в учебных материалах он сохранён как первый проход. Подключать каждый импортируемый файл отдельным script не нужно: браузер загрузит зависимости main.js.

type="module" нужен для import/export. Модульный script без async выполняется после разбора HTML, поэтому отдельный window.onload здесь не требуется. Модули выполняются в строгом режиме; переменные объявляем явно.

В HttpListenerStepByStep.csproj замените существующий ItemGroup копирования статики этим блоком:

```
<ItemGroup>
  <None Update="public\**\*.*">
    <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
  </None>
</ItemGroup>
```

Прежний public*.* охватывал только верхний уровень. Новый шаблон включает js и другие подпапки. Это инструкция для учебной копии проекта: исходный .csproj при создании туториала не изменялся.

16.7. Проверяем модули и разбираем ошибки

Перенесите содержимое examples-modules в HttpListenerStepByStep/public, поправьте шаблон .csproj, затем выполните dotnet build и dotnet run из каталога проекта. Открывайте страницу через http://localhost:5000/, а не file://.

1. В Network найдите /js/main.js и все шесть зависимостей. Каждый файл должен иметь статус 200 и JavaScript MIME-тип.
2. В Console нет SyntaxError и ошибок импортов. Создание стикера происходит один раз на клик.
3. Повторите проверки текста, Escape/Ctrl+Enter, drag со смещением/прокруткой, resize и одной ручки из раздела 15.
4. Убедитесь, что после dotnet build в bin/Debug/net10.0/public/js присутствуют все семь файлов.

Текущий Program.cs уже выдаёт .js как application/javascript; MIME-ветку менять не нужно. Но public ищется относительно рабочей папки. Для проверки скопированных файлов перейдите в bin/Debug/net10.0 и запустите dotnet HttpListenerStepByStep.dll; прежде остановите предыдущий сервер на том же порту.

Типичные ошибки:

- «Cannot use import statement outside a module»: в HTML остался обычный script.
- «does not provide an export named ...»: имя импорта и экспорт не совпали или забыли export.
- 404: неправильный путь/регистр, пропущено расширение .js или не скопирована подпапка js.
- Ошибка MIME: вместо JS сервер вернул HTML/текст ошибки. Проверьте статус и тело ответа.
- Два стикера после клика: одновременно подключены script.js и main.js.
- «board is not defined»: после переноса осталось обращение к прежней глобальной переменной. Передайте board аргументом.

Самопроверка: объясните, почему нет циклических импортов, где хранится активная модель и почему sticker-view не выполняет HTTP-запросы. Добавьте новую функцию в model.js и импортируйте её только там, где она нужна.

Учебный коммит: «Разделить frontend на ES-модули через export/import». Сборщик, npm и новые C#-пакеты для этого не требуются.

17. Как продолжить по исходному плану

В шагах 2-5 добавляем PostgreSQL, ограничения, функции/процедуры, репозиторий и HTTP API. Текущий пример сохраняет все пять изменяемых полей в одном объекте, что позволяет не переписывать геометрию и редактор при подключении БД.

Разделение из шага 6 уже выполнено в разделе 16. В дальнейшем расширяем api.js для HTTP-сохранения, а board-storage.js добавляем при изучении localStorage. Остальные обязанности модулей сохраняем.

В шагах 7-8 модель доски загружаем с сервера; локальные id заменяем UUID из ответов. Создание сначала сохраняем через POST, потом отображаем. При восстановлении используем готовый текст из БД, не запрашиваем новую цитату.

В шаге 9 handleStickerChange вызывает PUT с полным toStickerPayload. Пока запрос выполняется, контроллер блокирует действия над этим стикером; при ошибке возвращает последнюю подтверждённую модель. Текущий callback в консоли ещё не реализует это поведение.

Источники и границы материала:

- Учебный план HttpListenerStepByStep_PostgreSQL_Lesson_Plan.pdf: страницы 3-5, 11-14. Инструкции внутри PDF использованы как содержание плана, а не как команды на изменение проекта.
- Исходники: HttpListenerStepByStep/public/index.html, styles.css, script.js; Program.cs; HttpListenerStepByStep.csproj.
- MDN getBoundingClientRect: координаты окна и внешние размеры элемента.
- MDN clientLeft: толщина левой границы.
- MDN textContent: запись текста в узел без HTML-разметки.

https://developer.mozilla.org/en-US/docs/Web/API/Element/getBoundingClientRect

https://developer.mozilla.org/en-US/docs/Web/API/Element/clientLeft

https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

Полные примеры приложены для переноса в учебную копию проекта. Существующие исходники приложения при подготовке материалов не изменялись.